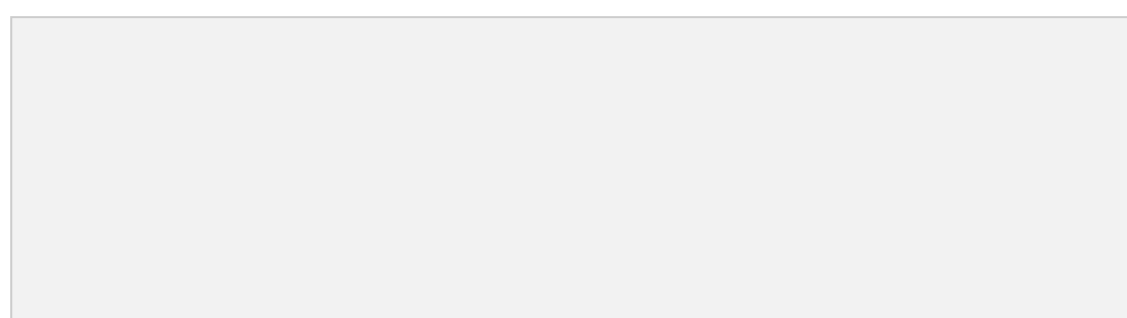




Internship Dashboard Design Document

1. Introduction

1.1 Purpose of the Design Document

This Design Document provides a comprehensive blueprint for the development of the Internship Dashboard system. It defines the architecture, components, data flow, database schema, API structure, and security mechanisms.

The purpose is to:

- Guide developers during implementation
- Ensure consistency in system design
- Serve as a reference for stakeholders and evaluators
- Support academic documentation (60-page report level)

1.2 System Overview

The Internship Dashboard is a full-stack web application designed to streamline internship workflows between Students and Mentors.

Key Features:

- Role-based dashboards (Student / Mentor)
- Task assignment & tracking
- Task submission system
- Progress monitoring
- Authentication & authorization

2. 🏗️ High-Level Design (HLD)

2.1 System Architecture

The system follows a MERN Stack Architecture:

Architecture Layers:

1. **Frontend (React.js):** User Interface, Dashboards (Student/Mentor), API communication
2. **Backend (Node.js + Express.js):** REST APIs, Business logic, Authentication
3. **Database (MongoDB):** NoSQL storage, Collections-based data model

Interaction Flow:

[User] → [React Frontend] → [API (Express)] → [Database (MongoDB)] → Response

2.2 System Components

Component	Technology	Role
Frontend	React.js	User Interface, Component rendering
Backend	Node.js + Express.js	API requests, Business logic, Authentication
Database	MongoDB	Stores Users, Tasks, Submissions

◆ Frontend

- Built using React.js
- Components:
 - Login/Register
 - Dashboard
 - Task List
 - Submission Form

◆ Backend

- Node.js + Express.js
- Handles:
 - API requests

- Business logic
- Authentication

◆ Database

- MongoDB
- Stores:
 - Users
 - Tasks
 - Submissions

2.3 Data Flow

Example Flow (Task Submission):

1. Student logs in
2. Fetch tasks from API
3. Student submits task
4. Backend validates data
5. Data stored in MongoDB
6. Mentor reviews submission

Request-Response Flow: Frontend $\rightarrow$ API Request $\rightarrow$ Controller $\rightarrow$ Database $\rightarrow$ Response $\rightarrow$ UI Update

2.4 Deployment Overview

Environment	Frontend	Backend	Database
Local Deployment	localhost:5173	localhost:5000	Local MongoDB instance
Production Deployment	Vercel / Netlify	Render / AWS	MongoDB Atlas

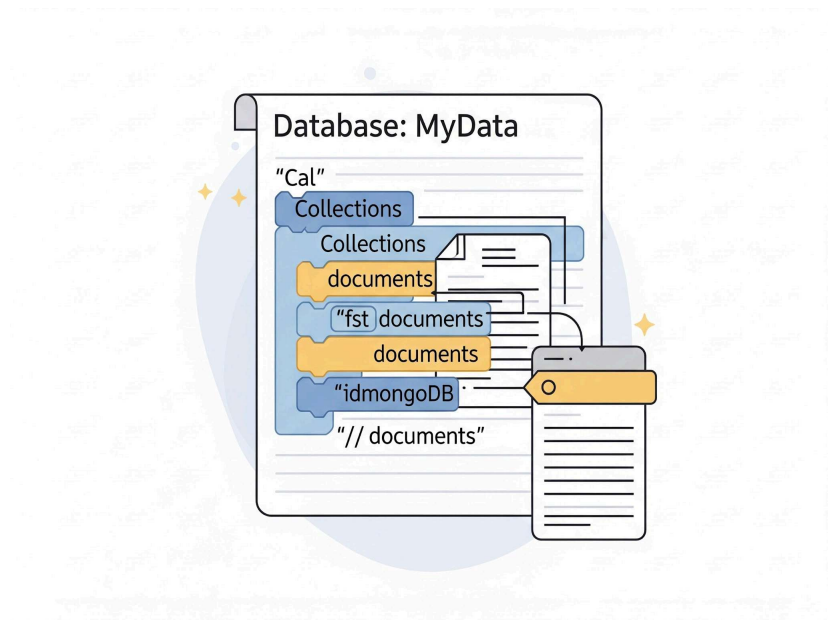
3. ⚙️ Low-Level Design (LLD)

3.1 Database Design

The system uses a NoSQL database, MongoDB, structured into three main collections.

📊 Collections Overview

Collection	Description
Users	Stores student & mentor data
Tasks	Stores assigned tasks
Submissions	Stores submitted work



Users Schema{

```
_id: ObjectId,  
  
name: String,  
  
email: String,  
  
password: String,  
  
role: "student" | "mentor",  
  
createdAt: Date  
  
}
```

Tasks Schema{

```
_id: ObjectId,  
  
title: String,  
  
description: String,  
  
assignedTo: ObjectId,  
  
assignedBy: ObjectId,  
  
deadline: Date,  
  
status: "pending" | "completed"  
  
}
```

Submissions Schema{

```
_id: ObjectId,  
  
taskId: ObjectId,  
  
studentId: ObjectId,  
  
submissionLink: String,  
  
remarks: String,  
  
submittedAt: Date  
  
}
```

3.2 API Design

The backend exposes RESTful APIs for communication with the frontend.

Authentication APIs

Endpoint	Method	Description
/api/auth/register	POST	Register user
/api/auth/login	POST	Login user

```
Example Login Request:{

  "email": "user@mail.com",

  "password": "123456"

}
```

 Task APIs

Endpoint	Method	Description
/api/tasks/create	POST	Assign task
/api/tasks	GET	Get tasks (filtered by user role)
/api/tasks/:id	GET	Get specific task

 Submission APIs

Endpoint	Method	Description
/api/submissions	POST	Submit task
/api/submissions/:taskId	GET	Get submissions for a task (Mentor view)

3.3 Module Design

Module	Primary Users	Key Functions
 Student Module	Students	View assigned tasks, Submit tasks, Track progress
 Mentor Module	Mentors	Assign tasks, Review submissions, Monitor performance
 Dashboard Module	All	Analytics, Task summaries, Progress charts

3.4 Component Design (Frontend)

React Structure:src/

├─ components/

├─ pages/

├─ services/

├─ context/

Key Components:

- Login.jsx
- Register.jsx
- StudentDashboard.jsx
- MentorDashboard.jsx
- TaskCard.jsx

State Management: Context API / Redux (optional)

API Integration: Axios for HTTP requests

3.5 Backend Logic

Route Handling:

- /routes/authRoutes.js
- /routes/taskRoutes.js
- /routes/submissionRoutes.js

Middleware:

- JWT Authentication
- Role-based Access Control

Controller Example:exports.createTask = async (req, res) => {

const task = await Task.create(req.body);

res.json(task);

};

4. Security Design

Security is implemented at multiple layers to protect data and control access.

- **Authentication:** JWT-based Authentication for stateless session management.
- **Password Storage:** Password hashing using bcrypt to secure user credentials.
- **Authorization:** Role-based access control to ensure users (student/mentor) only access authorized endpoints and data.
- **Protected Routes:** Middleware is used to secure all relevant API routes, preventing unauthenticated access.

5. Error Handling & Validation

Robust error handling and input validation are essential for system stability.

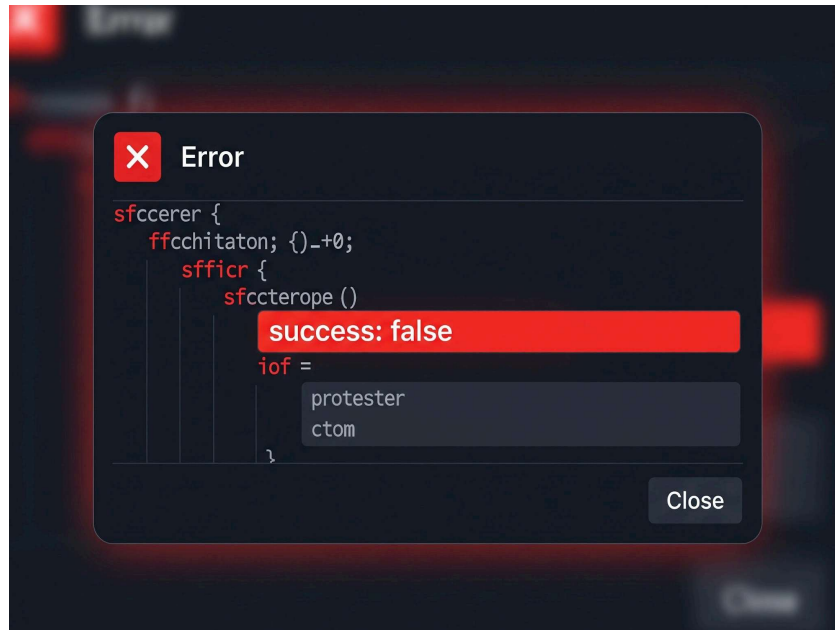
Input Validation:

- Required fields check for all forms.
- Email format validation on registration/login.
- Data type validation (e.g., ensuring `deadline` is a Date).

API Error Responses

When an error occurs, the API will return a standard JSON response structure:

```
"success": false,  
  
"message": "Invalid credentials"  
  
}
```

6. 📌 Conclusion

The Internship Dashboard design follows a scalable and modular architecture using the MERN stack. The system ensures:

- Efficient interaction between students and mentors
- Secure authentication
- Clean separation of frontend and backend
- Scalable database structure

